

Using HTMX To Make Interactive Elements In Drupal

DrupalCamp England 2026

Philip Norton

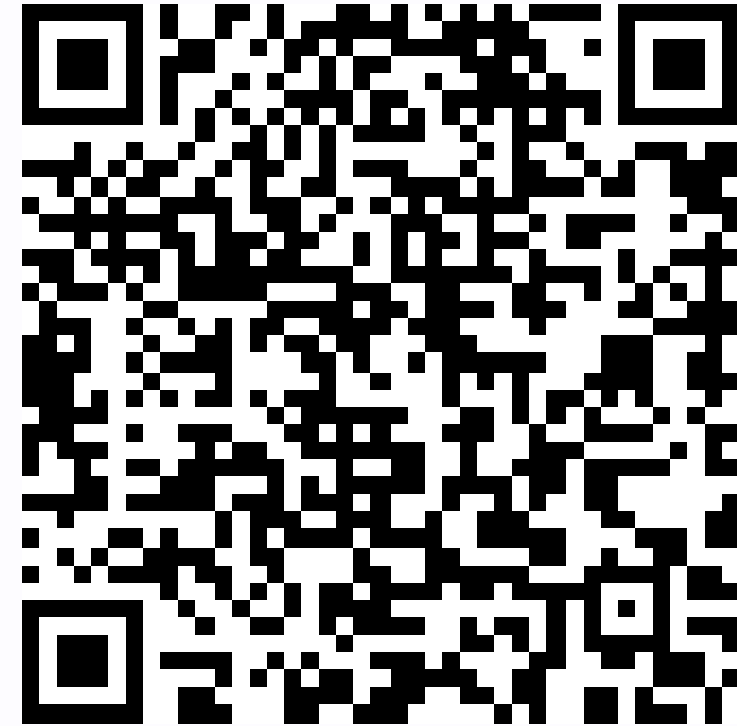
- Developer at Code Enigma
- Involved with Drupal for 20 years
- Owner of `#!` code
(www.hashbangcode.com)

Philip Norton hashbangcode.com fosstodon.org@hashbangcode
fosstodon.org@philipnorton42



Source Code

- Talk is available at:
<https://github.com/hashbangcode/drupal-hmx-talk> or via QR code
- All code seen can be found at:
 - <https://bit.ly/46arnzj> HTMX examples
 - <https://bit.ly/3OHAIIR> Drupal examples
- More resources at
www.hashbangcode.com



Using HTMX To Make Interactive Elements In Drupal

HTMX In Drupal

- HTMX added to Drupal in 11.3 as a core component.
- This is the new standard for ajax requests.
- All existing ajax features will be re-written in HTMX.

HTMX

What is HTMX?

- JavaScript framework.
- Allows ajax calls and CSS Transitions *without writing JavaScript code*.
- Small at 16K (minified and gzipped).
- No other dependencies.
- Platform agnostic.
- Plugins extend functionality.

What is HTMX?

- Behaviour is added to HTML attributes.
- Any element can issue a web request.
- All responses should be in HTML.

Installing HTMX

Download and include the single JavaScript file.

```
<script src="htmx.min.js"></script>
```

There are CDN options available.

We are now ready to use HTMX.

HTMX Attributes

- Prefixed with `hx-` or `data-hx-`.
- 11 attributes are commonly used.
- There are another 20-30 that add more functionality.

HTMX Attributes - Example

```
<button hx-get="index.php" hx-target="#div1">Submit</button>  
  
<div id="div1"></div>
```

This example will:

- `hx-get` = Send a "get" request to the index.php path.
- `hx-target` = Write the response in the element with the ID "div1".

HTMX Requests

- All requests from HTMX will have the `hx-request` header.
- Certain attributes will add further headers to the request.

Responding To The Request

Your response should be in plain HTML.

We could respond to the previous request with:

```
<p>Button clicked!</p>
```

Which would be injected into our page like this.

```
<div id="div1"><p>Button clicked!</p></div>
```

Responder Class In PHP

A simple class to respond to HTMX requests.

```
class Htmx {  
    public static function isHtmxRequest(): bool {  
        return isset($_SERVER['HTTP_HX_REQUEST']) && $_SERVER['HTTP_HX_REQUEST'] === 'true';  
    }  
  
    public static function isGet(): bool {  
        return $_SERVER['REQUEST_METHOD'] === 'GET';  
    }  
}
```

Responding To The Request

We detect the `hx-request` header and the correct HTTP method before responding.

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<p>Button clicked at ' . date('r') . ' .</p>';  
}
```

The div now looks like this:

```
<div id="div1"><p>Button clicked at Sun, 08 Feb 2026 16:00:23 +0000.</p></div>
```

HTMX Common Attributes

HTTP Verbs

Different HTTP verbs are available through attributes.

- `hx-get` does a "get" request.
- `hx-post` does a "post" request.
- `hx-delete` does a "delete" request
- `hx-patch` does a "patch" request
- `hx-put` does a "put" request

hx-target

The target element to be swapped.

```
<button hx-get="index.php" hx-target="#div1">Submit</button>  
<div id="div1"></div>
```

hx-trigger

Defines the action that will trigger the request.

```
<div hx-post="index.php" hx-trigger="click">Click me</div>
```

- Can be left out for elements that have default triggers.
- Eg. `<button>` elements automatically have a `click` trigger.

hx-trigger Examples

```
<div hx-post="index.php" hx-trigger="click">Click me</div>
```

```
<div hx-post="index.php" hx-trigger="click once">Click me</div>
```

```
<div hx-post="index.php" hx-trigger="revealed"></div>
```

```
<div hx-post="index.php" hx-trigger="load delay:500ms"></div>
```

```
<input hx-get="index.php" hx-trigger="keyup delay:1s" />
```

hx-select

Select content to swap with the target element from the response.

HTMX:

```
<button hx-get="index.php" hx-select="#response" hx-swap="outerHTML">Click</button>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<p id="response">Button clicked at ' . date('r') . ' .</p>';  
    echo '<p>Some extra content that won\'t get displayed.</p>';  
}
```

hx-select-oob

Select an "Out Of Band" element to also target in the response.

HTMX:

```
<button hx-get="index.php" hx-select-oob="#div1">Send Request</button>  
<div id="div1"></div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo 'Request Sent';  
    echo '<div id="div1">Button clicked at ' . date('r') . '</div>';  
}
```

hx-select-oob - Continued

You can comma separate this property to do multiple things.

HTMX:

```
<button hx-get="index.php" hx-select-oob="#div1,#div2">Send Request</button>  
<div id="div1"></div> <div id="div2"></div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo 'Request Sent';  
    echo '<div id="div1">Button clicked at:</div>';  
    echo '<div id="div2">' . date('r') . '</div>';  
}
```

hx-swap

- Controls how an element will be swapped into the page.
- Defaults to `innerHTML` - replace the HTML contents.
 - `outerHTML` - Replace the element.
 - `textContent` - Replace contents without parsing as HTML.

hx-swap - Continued

- `beforebegin` , `afterbegin` , `beforeend` , `afterend` - Place the response before or after the element and its children.
- `delete` - Delete the target element from the page.
- `none` - Do nothing with the response (but still process out of band items).

hx-swap-oob

Swap an "Out Of Band" element. Similar to `hx-select-oob` but the response tells HTMX what to swap.

HTMX:

```
<button hx-get="index.php" hx-swap="none">Send Request</button>  
<div id="div1"></div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<div id="div1" hx-swap-oob="true">Button clicked at ' . date('r') . '</div>';  
}
```

CSS Transitions

- HTMX adds classes to elements during different events.
- `htmx-added` is added to new content before it is added to the page.
- We can use these classes to animate what HTMX is doing with CSS transitions.

CSS Transitions

The following will fade in a bit of content inside the `div1` element as it is loaded into the page.

```
#div1 p {  
    opacity: 1;  
    transition: opacity 1s ease-in;  
}  
#div1 p.htmx-added {  
    opacity: 0;  
}
```

Configuring HTMX

- It is possible to configure HTMX via a number of settings.
- Can be set through meta tags (preferred)

```
<meta name="htmx-config" content='{ "defaultSwapStyle": "outerHTML" }'>
```

Or JavaScript.

```
<script>  
htmx.config.defaultSwapStyle = 'outerHTML';  
</script>
```

DEMO!

HTMX In Drupal

HTMX In Drupal

- HTMX is included in Drupal as a library.
- A few classes exist to assist with adding attributes and responding to requests.

HTMX in Drupal

The standard usage of HTMX in Drupal is to use the `data-hx-` prefix for attributes.

Eg:

```
<button data-hx-get="index.php" data-hx-target="#div1">Submit</button>  
  
<div id="div1"></div>
```

Drupal Libraries

HTMX can be included into the page through a Drupal library.

- `core/htmx` - The core HTMX library.
- `core/drupal.htmx` - Additional scripts for Drupal.
- You rarely need to inject these libraries directly.

Drupal Libraries

The library `core/drupal.htmx` includes three files:

- `htmx-assets.js` - Adds assets the current page requires.
- `htmx-behaviors.js` - Connect `Drupal.behaviors` to HTMX inserted content.
- `htmx-utils.js` - Helper functions for the other two files.

Drupal Integration

- The `\Drupal\Core\Htmx` class is used as a wrapper around the HTMX libraries and attribute injection for render arrays.
It can also inject headers for responding to HTMX.
- `\Drupal\Core\Htmx\HtmxRequestInfoTrait` is used to detect HTMX requests and associated headers.

Htmx Class

- Has a number of methods that set up attributes.

```
$output['element'] = [  
  '#type' => 'html_tag',  
  '#tag' => 'p',  
  '#value' => 'Content'  
];  
  
$htmx = new Htmx();  
$htmx->get()  
  ->swap('afterend')  
  ->trigger('revealed')  
  ->applyTo($output['element']);
```

```
<p  
  data-hx-get="/route"  
  data-hx-swap="afterend"  
  data-hx-trigger="revealed">  
Content  
</p>
```

Htmx Class

- The HTMX class can also be used with a different syntax.

```
(new Htmx())  
  ->get()  
  ->swap('afterend')  
  ->trigger('revealed')  
  ->applyTo($output['element']);
```

HTMX Drupal Response

- Drupal has the ability to respond to a HTMX response with HTML.
- This uses the class `Drupal\Core\Render\MainContent\HtmxRenderer`.
- Renders the render array as a little HTML document and sends this upstream.

HTMX Drupal Response

HtmxRenderer is invoked by:

- `_wrapper_format=drupal_htmx` as a query on the incoming request.
- The `_htmx_route=true` on the route replying to the request.

HTMX With Controllers

HTMX With Controllers

- Think about your approach.
 - Do you have two actions in your controller or one action that serve normal and one for HTMX requests?
- Two actions means you need to tag one route with the route option `_htmx_route: TRUE`.
- One route means you need to add some code to your controller footprint to react to HTMX requests.

HTMX With Controllers

The `__htmx_route: TRUE` is used to create the HTMX response action.

```
mymodule_controller_action_htmx:  
  path: '/htmx-response'  
  defaults:  
    _controller: '\Drupal\mymodule\Controller\MyModuleController::htmx'  
  requirements:  
    _permission: 'access content'  
  options:  
    _htmx_route: TRUE
```

HTMX With Controllers

The `HtmxRequestInfoTrait` trait needs access to the request stack service.

```
namespace Drupal\mymodule\Controller;

use Drupal\Core\Controller\ControllerBase;
use Drupal\Core\Htmx\HtmxRequestInfoTrait;

class MyController extends ControllerBase {

    use HtmxRequestInfoTrait;

    public function __construct(protected RequestStack $requestStack) {}

    public function action() {}
}
```

HTMX With Controllers

Then, in your action:

```
public function action() {  
    if ($this->isHtmxRequest()) {  
        // Respond to HTMX request.  
    }  
  
    // Respond to normal controller action.  
    // Generate markup to set up the HTMX request.  
}
```

HTMX With Forms

HTMX With Forms

- The trait `HtmxRequestInfoTrait` is part of the `FormBase` class.
- Important!
 - The HTMX response in Forms is the entire form, so you *need* add a `hx-select` (or similar) to pick out the relevant part of the response.
 - Also, forms need to be consistent. You can't just throw elements into the form markup as the elements need to exist in the form build.

HTMX With Forms

Forms in the same way as constructors, you just need to decorate the elements in question.

```
(new Htmx())  
  ->post()  
  ->target('*:has(>input[name="email"])')  
  ->select('*:has(>input[name="email"])')  
  ->trigger('keyup delay:1s')  
  ->applyTo($form['email']);
```

- This decorates the form element with the HTMX elements.

DEMO!

Drupal Needs You!

- HTMX is in Drupal 11.3 as a fully featured system.
- All of the ajax subsystem now needs to be converted to HTMX.
- There's a lot of work to be done.
- <https://www.drupal.org/community-initiatives/replace-ajax-api-with-htmx>

HTMX Module

- <https://www.drupal.org/project/htmx>
- Some extra tools and examples of HTMX in a Drupal setting.
- Has a Views plugin to display Views as HTMX.
- Has a HTMX debugger that logs all HTMX events in the browser console.

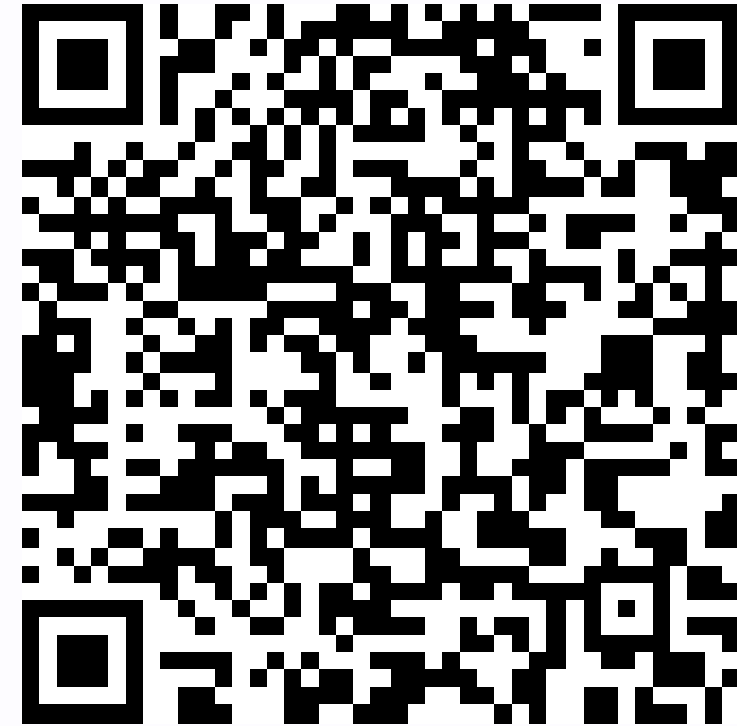
Resources

- htmx.org - <https://htmx.org>
- [HTMX Labs](https://htmxlabs.com) - <https://htmxlabs.com>

Questions?

- Slides:

<https://github.com/hashbangcode/drupal-hmx-talk>



Thanks!

- Slides:

<https://github.com/hashbangcode/drup-hmx-talk>

